Clothing Store Recommendation System

Data Structures and Advanced Algorithms Final Report

Ema Martins - up202402794
Tomás Ribeiro - up202007710

June 10, 2025

Contents

1 Introduction

1.1 About our problem

Nowadays, almost everything is done online, and shopping for clothes is no exception. It's been growing rapidly in recent years.

Many algorithms, most based on artificial intelligence, have been developed to provide users with accurate and personalized recommendations. While these models can be highly effective, they often operate as black boxes, making it difficult to understand the patterns they use. This lack of transparency also makes it challenging to introduce an element of randomness, which is essential for showing users a variety of products and encouraging broader discovery.

For this reason, we decided to develop a recommendation system based on a Kd-tree structure. This approach allows us to explicitly control the recommendation patterns and apply weights based on criteria we define, giving us both transparency and flexibility in how we suggest products to users.

2 Contextualization

2.1 How kd-trees work

Kd-trees organize data similarly to binary trees. They **split the space based on one of the dimensional axes** (x, y, z, ...) using one of the following strategies:

- **Alternate between dimensions sequentially.**
- **Pick the dimension with the highest variance.**

Then, the points are sorted along the chosen axis, and the **median point is selected as the reference node** to split the space. Using the median helps balance the tree, enabling efficient search operations.

Two subtrees are then created. The **left subtree** contains the points with **values smaller** than the median along the chosen axis, while the **right subtree** contains the points with **values greater** than the median.

This process is repeated recursively for each subset, **switching axes at each level**. It **stops when no points remain**.

2.2 How to retrieve similar products

Our **first approach** was to **create clusters based on certain criteria**, allowing us to navigate between clusters when it was necessary to display different products. However, this resulted in poor performance because it was difficult to define a consistent and meaningful way to form these clusters.

We then considered **alternative solutions** and decided to develop a euclidean distance-based weighting function which we could use to define the criteria with which similar products were chosen. This, in unison with the KD-Tree's fast nearest neighbour search, allowed us to develop a fast, effective and customizable way of retrieving similar products.

3 Implementation

3.1 Dataset

We used a dataset constructed by **merging two original datasets**, retaining only the attributes deemed relevant to our project. The final structure of this dataset is organized and presented across three tables: Table 1 shows the basic product information, Table 2 details the style and usage context, and Table 3 provides information related to pricing, brand, and discount status.

id	gender	masterCategory	subCategory	articleType
15970	Men	Apparel	Topwear	Shirts
39386	Men	Apparel	Bottomwear	Jeans
59263	Women	Accessories	Watches	Watches
21379	Men	Apparel	Bottomwear	Track Pants

Table 1: Basic Product Information

id	baseColour	season	year	usage	ageGroup
15970	Navy Blue	Fall	2011	Casual	Adults-Men
39386	Blue	Summer	2012	Casual	Adults-Men
59263	Silver	Winter	2016	Formal	Adults-Women
21379	Black	Fall	2011	Casual	Adults-Men

Table 2: Style and Usage Context

id	price	discountedPrice	brandName	has_discount
15970	1195.0	1195.0	Turtle	0
39386	1499.0	1300.0	Peter England	1
59263	6500.0	6500.0	Titan	0
21379	1699.0	1699.0	Manchester United	0

Table 3: Price, Brand, and Discount Details

The only column that was not originally present in either dataset is **has_discount**. It was **generated** by comparing the price and discountedPrice values to determine whether a product is on sale.

Based on the product ID, we have a **directory containing the product images**, where each image file is named after the corresponding ID. This provides a straightforward way to retrieve the images.

3.2 Data preparation

The **price attribute**, which is a **continuous numerical variable, was discretized into five categories** using pandas.cut. This step transformed raw prices into labeled ranges (Case 1, Case 2, ..., Case 5), allowing the model to generalize better and **reducing the influence of extreme values or outliers**. This making the feature space more balanced and suitable for similarity-based computations.

To make the categorical attributes usable by the recommendation algorithm, we applied **one-hot encoding** using OneHotEncoder. This technique transforms each categorical value (gender, masterCategory, subCategory, articleType, baseColour, season, usage, price, brandName, and ageGroup) into a binary vector, where **each unique category is represented by a separate feature**. This encoding ensures that the categorical data is **converted into a numerical format without imposing any ordinal relationship among the categories**. The problem with this approach is that the twelve original features that we had were converted into hundreds of binary features. The reason this is a problem is that it leads to a dramatic increase in dimensionality, resulting in a sparse and high-dimensional feature space. This severely affects the performance of KD-Trees, which rely on distance metrics to find nearest neighbors. In high-dimensional spaces, distances between points become less informative, and all points tend to appear similarly distant from each other. Additionally, since one-hot encoding treats all categories as equidistant, the KD-Tree cannot capture any underlying semantic relationships between categories, which once again affects the quality of the recommendations.

In the next section, we address the steps that were taken in order to minimize this fundamental problem of the KD-Tree.

3.3 KD-Tree

Originally, we planned to implement a classical KD-Tree where each node represented one product in the dataset, and the products would be split on some defined features, but in this implementation, we quickly realized that the tree was being fully run through in each recommendation search. This prompted us to make several modifications to the way that the tree works.

```
class KDNode:
    function init( points, split_axis, left=None, right=None):
        self.points = points
        self.axis = axis
        self.left = left
        self.right = right
```

Figure 1: KDNode pseudocode

```
def build_kdtree(points, depth = 0, split_axis_order = None, max_depth = 15, min_bucket_size = 10):

    if depth >= max_depth or len(points) <= min_bucket_size:
        return KDNode(points, None)

    axis = split_axis_order[depth]

    left_points = [p for p in points if p[0][axis] == 0]
    right_points = [p for p in points if p[0][axis] == 1]

    return KDNode(
        points,
        axis,
        build_kdtree(left_points, depth + 1, split_axis_order, max_depth, min_bucket_size),
        build_kdtree(right_points, depth + 1, split_axis_order, max_depth, min_bucket_size)
    )
```

Figure 2: KD-Tree build pseudocode

We made it so the tree starts by splitting on the most discriminative features, prioritizing those that contribute the most to meaningful product differentiation. An ideal split is one that splits the products in half so that one half goes to one side of the tree and the other half goes to the other side of the tree, allowing for a more balanced tree. Since one-hot encoding resulted in an excessive number of binary features, we imposed a constraint on the tree's height to prevent unnecessary splits and reduce computational overhead. Rather than allowing the tree

to expand fully, we stopped splitting once we observed that the leaf nodes were yielding sufficiently relevant recommendations. To further improve recommendation quality, we applied a personalized Euclidean-based weighting function, the criteria for which are detailed in the next section. Additionally, instead of having each leaf node represent a single product, we modified the structure so that each leaf stores a bucket of products that reached that point in the tree, which was necessary given that the tree's height was limited.

3.4 Recommendations Search

```
def recommendation_search(root, target, k, weights, heap, number_of_backtracks):

    if root.left is None and root.right is None:
        for point, idx in root.points:
            dist = sqrt(sum(weights * (target - point)**2))
            heapq.heappush(heap, (-dist, idx, point))
            if len(heap) > k:
                heapq.heappop(heap)
        return sorted([(-d, idx) for d, idx, _ in heap])

    axis = root.axis

    if target[axis] == 0:
        close_node = root.left
        away_node = root.right
    else:
        close_node = root.right
        away_node = root.left

    knn_search(close_node, target, k, weights, heap, number_of_backtracks)

    if len(heap) < k or number_of_backtracks[0] > 0:
        number_of_backtracks[0] -= 1
        knn_search(away_node, target, k, weights, heap, number_of_backtracks)

    return sorted list of (-d, idx) from heap
```

Figure 3: Recommendations Search pseudocode

So a client selects a product, the way that this method works is, the selected product will go down the tree, being **split on the most discriminative features**, when it reaches its respective leaf node, it will be among all the other products, minimum ten, that were split on the exact same features. At this point, it is already among products similar to itself, so the selected product is compared with these products using a weighting function. A max heap is used in order to keep

track of the best recommendations throughout the algorithm. This could have easily been switched with an ordinary list, but a max heap simply performed far better for this specific task due to its efficient handling of dynamic top-k elements.

After the selected product passes through its leaf node, it **starts backtracking and visiting the nearby branches**. For this we used a parameter number_of_backtracks, which defines the number of times that, when backtracking, an unexplored branch would be explored. This was necessary, once again, due to the high number of binary features which didn't allow for the correct evaluation of an unexplored branch being worth visiting. This happens because all one-hot encoded values for a given feature are equidistant from one another, regardless of their actual semantic similarity.

After the number_of_backtracks reaches 0, backtracking ends and the algorithm stops with the top k recommendations found.

3.5 Criteria

We analyze the attributes available in the dataset and compare each one with the selected (reference) product. We assign weights to all attributes, except for id, based on how well they match the reference.

If a product **shares the same category** as the chosen one, we assign a **weight of 1** for that attribute. Otherwise, it receives a weight of 0.

Next, we handle **color matching**, based on the baseColour attribute. If the product has the **exact same color** as the reference, we assign a **weight of two**. If the color is different but considered a **good combination** according to a predefined color-matching dictionary, we assign a **weight of one**. If there's no match or compatibility, the weight remains 0.

Another important factor is whether the product **is on sale**. If a product is on sale (column has_discount is 1), it automatically receives a **weight of 1**, even if it's otherwise quite different from the reference product. This **introduces a bit of randomness** to encourage more diverse recommendations.

Additionally, we give a **weight of two** to products that **match the same subCategory** (such as watches, topwear ...) also to those with the **same intended usage** (like casual or sporty styles) when they are similar to the selected item.[2]

4 Balance of Parameters

Some parameters such as the **depth of the tree**, the **number of products per bucket**, and the **number of backtracks** significantly influence the number of products we evaluate to determine whether they are suitable. We needed to find a balance that avoids visiting too many products, since that would increase computation time, while still exploring a sufficient number of diverse products. This diversity is important because products within the same bucket tend to be very similar.

As shown in Figure 4, the number of products compared increases with the bucket size. This behavior is expected because with a larger bucket size, and keeping the depth and number of backtracks constant, more products fall within each bucket, resulting in more comparisons.

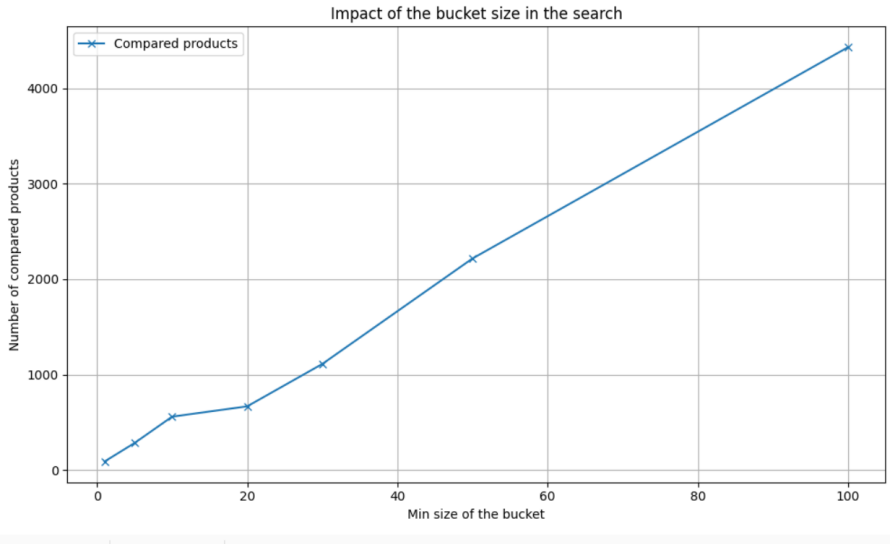

Figure 4: Influence of bucket size

Figure 5 shows how the tree depth influences the results. With only 2 levels, the data remains grouped closely, limiting exploration. In contrast, with 12 or more levels, the data is more spread out, requiring a broader search across the dataset.

However, after a certain point (e.g., depth greater than 12), the data is already well distributed across buckets, so increasing the tree depth further does not lead to significant improvements.

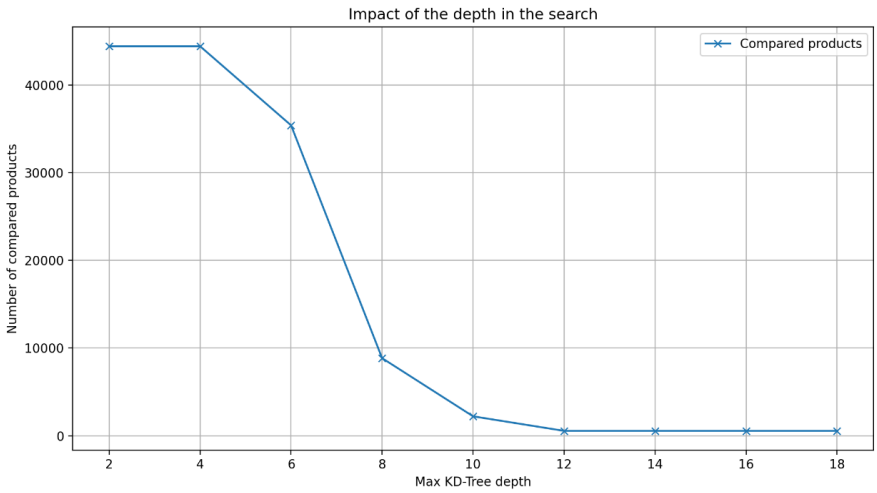

Figure 5: Influence of tree depth

Figure 6 presents a linear regression that confirms our expectations. Since most buckets are filled uniformly (i.e., contain the same number of products), increasing the number of backtracks leads to a proportional increase in the number of visited buckets, and thus, more product comparisons.

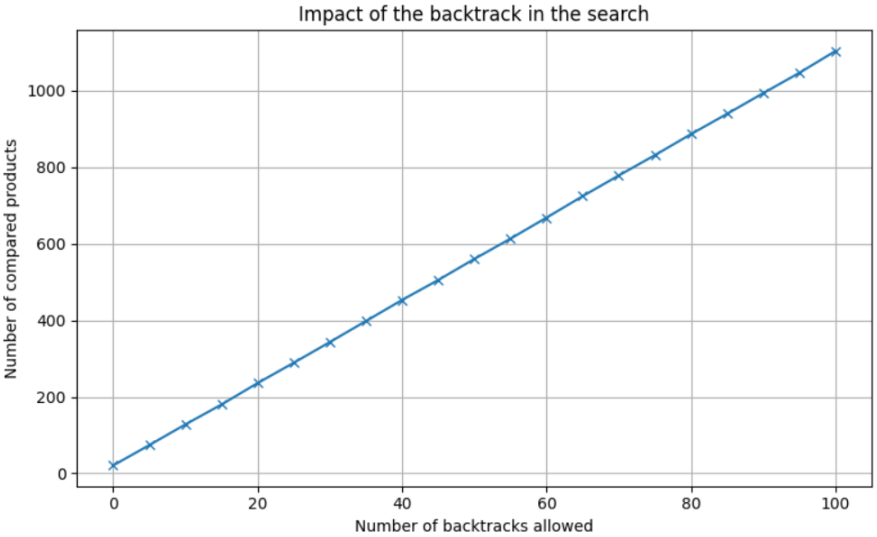

Figure 6: Influence of the number of backtracks

Based on this analysis, we determined that a good balance between the number of comparisons and the diversity of recommended products could be achieved with the parameter values shown in Table 4.

Parameter	Value
Depth of the tree	15
Products per bucket	10
Number of backtracks	50

Table 4: Parameters used in the system

5 The web application

The web application is built using Python [1] Flask for the backend and HTML, CSS and JavaScript for the frontend.

5.1 Paginated Product Gallery

One of the key features of the application is the paginated product gallery that shows 5 clickable products per page as shown in Figure 7. For each product shown, the ID, the price and the discount, if available, are presented.

Figure 7: Web Application UI

5.2 Product Recommendations

Upon clicking a product, the product clicked is enlarged, and 5 recommendations generated by our KD-Tree are shown with the associated product ID, price, and discount.

Figure 8: Web Application Product Recommendations

6 Performance

6.1 Empirical study data

To better understand whether our results are better or worse than those obtained on other websites, we conducted an empirical study.

In the first step, we asked to the participants to search for a product on our website that they would likely purchase. After they selected a product, we showed them five recommendations generated by our system. We then asked them to evaluate each recommendation as a good, medium, or bad suggestion.

To quantify the results, we assigned numerical values to the responses: **0 for bad, 0.5 for medium, and 1 for good. We calculated the average score for each product's set of recommendations and then computed the overall average to evaluate the performance of our recommendation system.** The obtained results are present in Table 5.

product id	recommendations medium
1	0.7
2	0.7
3	0.6
4	0.4
5	0.5
6	0.5
7	0.5
8	0.6
9	0.8
10	0.2
11	0.9
12	0.2

Table 5: Results of our recommendation system

To better understand how effective our recommendation system is, we conducted the same evaluation using recommendation systems from other platforms. Table 6 presents the results obtained for the top five recommendations of a product on SHEIN [4] and Table 7 presents the results obtained for the top five recommendations of a product on Primark [3].

product id	recommendations medium
1	0.4
2	0.4
3	0.4
4	0.6
5	0.7
6	0.8
7	0.6
8	0.5
9	0.6
10	0.4
11	0.4
12	0.2

Table 6: Results of Shein Recommendation System

product id	recommendations medium
1	0.5
2	0.6
3	0.8
4	0.8
5	0.6
6	0.8
7	0.4
8	0.4
9	0.4
10	0.7
11	0.4
12	0.3

Table 7: Results of Primark Recommendation System

After collecting all this data, we calculated the average of the averages of the recommendations for each system, as shown in Table 8.

system	medium performance
Our system	0.550
SHEIN's system	0.500
Primark's system	0.558

Table 8: Results of our recommendation system

6.2 Analysis of Results

After analyzing the results, we observed that our values are comparable to those obtained by large e-commerce platforms. A key advantage of our system is that we have full knowledge and control over the patterns and parameters used. With further research into fashion trends and user preferences, it would be relatively easy to fine-tune certain parameters to achieve even better performance.

We also noticed that, on major platforms, recommendation results are heavily influenced by users' previous searches. Over time, this influence becomes so strong that the recommendations lose diversity, making it difficult for users to discover new products they might like. In contrast, our recommendation system does not suffer from this issue. While our current model avoids overfitting to individual preferences, we could consider adding a small weight in the future for preferences

inferred from recent user behavior, striking a balance between personalization and discovery.

It's important to highlight that some of the suggestions we considered "good" in these large platforms were simply the same product in different colors or patterns. In our view, these do not represent high-quality recommendations, since users already have access to all available variations once they click on a product. Thus, recommending these variants adds little value.

One limitation in our study was not taking product prices into account during the evaluation. We only asked users whether they liked the suggestions, regardless of price. However, in our system, price plays a significant role, having a higher weight compared to other product features. This is something that should be better reflected in future evaluations.

It's also worth considering that large e-commerce platforms benefit from access to much larger datasets. This gives them a significant advantage in tailoring recommendations and meeting user needs more effectively.

Despite these limitations, we consider our results to be strong. Naturally, there is still room for improvement in our approach, but the final outcome meets the criteria we defined as indicative of good system performance.

7 Conclusions

After all our research, we conclude that a slightly improved structure can lead to the same or possibly better performance compared to the approaches that use AI.

The transparency of the results could be valuable not only for helping us understand user behavior and improve our product, but also for clothing platforms aiming to better understand their users' needs.

We believe there is still room for improvement. If we had more time, the next steps would include testing different weight combinations and assigning more importance to the most common characteristics users look for when making purchases. For this second approach, it would be necessary to create user accounts in order to track their interactions.

It will also be important to repeat the empirical study, this time asking users to take price into consideration during the exercise. Comparing these results with our previous study would help us assess how much influence price has on system performance.

In conclusion, we consider our project to have been well executed, given the time constraints and the requirements of the assignment. Check out the code on GitHub: ema-12-martins/EDAA.

8 References

References

[1] Python Software Foundation. *Welcome to Python.org*. Accessed on May 26, 2025. 2025. URL: https://www.python.org/.

[2] Evgenia Karpova, Elena Karpova, and Grace Kunz. *Development of Apparel Product Evaluation (APE) Framework*. Accessed on May 26, 2025. 2021. URL: https://libres.uncg.edu/ir/uncg/f/E_Karpova_Develpment_2021.pdf.

[3] Primark. *Beauty — Primark Portugal*. Accessed on May 26, 2025. 2025. URL: https://www.primark.com/pt-pt/c/beleza.

[4] SHEIN. *SHEIN Portugal — Women's, Men's, Kids' Fashion and More*. Accessed on May 26, 2025. 2025. URL: https://pt.shein.com/.